МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Работа с файлами

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ, работающих с текстовыми файлами.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Изучите теоретические сведения к данной лабораторной работе.
2. Ознакомиться с постановкой задачи (вариант задания вычисляется как ваш_номер_в_списке_группы % количество_вариантов).
3. Составить программу, решающую поставленную задачу. ***Все функции разместить в отдельном файле (модуле)***, который будет подключаться в основном проекте.
4. Отладить программу на подготовленных наборах тестовых данных.
5. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:
   – постановку задачи;
   – текст программы с комментариями;
   – наборы тестовых исходных данных с соответствующими результатами работы программы;
   – краткие ответы на контрольные вопросы
6. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Создать текстовый файл, в котором будут храниться элементы исходных списков. В отдельном модуле написать три функции:

1. Первой передаётся путь к текстовому файлу. Необходимо считать из файла элементы списка, преобразовать их к необходимому типу данных (int, float) и вернуть в качестве результата полученный список.

2. Вторая получает список в качестве параметра. Выполняет действия согласно варианту задания и возвращает результат.

3. Третья получает результат работы второй функции и путь к другому текстовому файлу. В указанный текстовый файл сохраняются результаты работы.

В основной программе связать все три функции в один рабочий код.

Вариант №5.

В списке целочисленных значений X[N] найти max – максимальный элемент среди четных элементов списка и min – минимальный элемент среди всех элементов списка.

ХОД РАБОТЫ

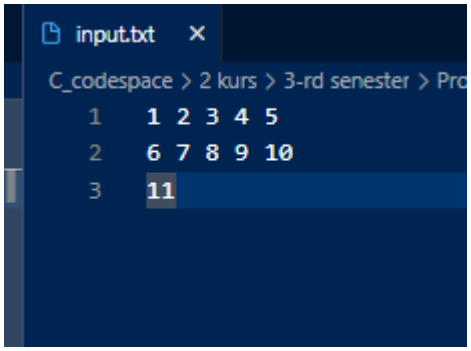
Файл **header.py**:

```python
def read_data_from_file(file_path):
    try:
        with open(file_path, 'r') as file:
            content = file.read() # Читаем содержимое файла
            numbers_str = content.split() # Разбиваем в список строк по пробелам
            numbers = [int(num_str) for num_str in numbers_str] # Преобразуем строки в числа
        return numbers
    except FileNotFoundError:
        print(f"ОШИБКА: Файл {file_path} не найден.")
        return
    except ValueError:
        print(f"ОШИБКА: Невозможно преобразовать данные в целые числа в файле {file_path}.")
        return


def process_data(data_list):
    if not data_list:
        print("ОШИБКА: Список пуст.")
        return

    min_val = min(data_list) # Находим минимальный элемент среди всех

    even_numbers = [num for num in data_list if num % 2 == 0] # Список чётных элементов

    if not even_numbers:
        print("В списке нет чётных чисел.")
        max_even = None
    else:
        max_even = max(even_numbers)

    return max_even, min_val


def write_result_to_file(result, file_path):
    max_even, min_all = result
    try:
        with open(file_path, 'w', encoding='utf-8') as file:
            if max_even is None:
                file.write("Максимальный чётный элемент: не найден\n")
            else:
                file.write(f"Максимальный чётный элемент: {max_even}\n")
            if min_all is None:
                file.write("Минимальный элемент: не найден\n")
            else:
                file.write(f"Минимальный элемент: {min_all}\n")

        print(f"Результаты успешно записаны в {file_path}")
    except Exception as e:
        print(f"ОШИБКА при записи в файл {file_path}: {e}")
```

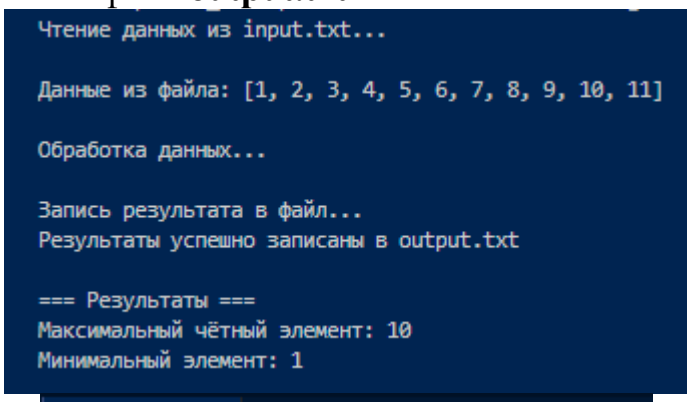
Файл **main.py**:

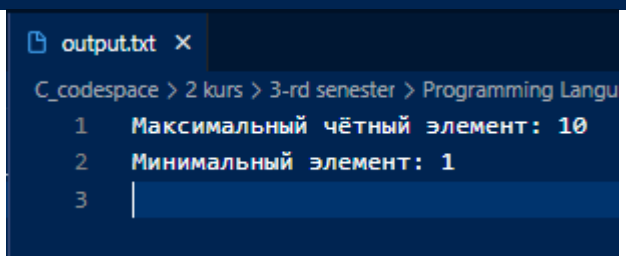
```python
import header

# Пути к файлам
input_file = "input.txt"
output_file = "output.txt"

print(f"Чтение данных из {input_file}...")
data = header.read_data_from_file(input_file)

if data:
    print(f"\nДанные из файла: {data}")

    print("\nОбработка данных...")
    result = header.process_data(data)

    print("\nЗапись результата в файл...")
    header.write_result_to_file(result, output_file)

    max_even, min_all = result
    print("\n=== Результаты ===")
    if max_even:
        print(f"Максимальный чётный элемент: {max_even}")
    else:
        print("Максимальный чётный элемент: не найден")
    if min_all:
        print(f"Минимальный элемент: {min_all}")
    else:
        print("Минимальный элемент: не найден")
else:
    print("ОШИБКА: Не удалось прочитать данные из файла или файл пуст.")
```

Файл **input.txt** с элементами исходных списков:



Чтение данных списка из файла, обработка полученных данных и сохранение данных обработки в файл **output.txt**:





ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Какие типы файлов существуют? В чём их особенность?**
   - Текстовые — содержат символы, читаемые человеком (.txt, .csv, .py и др.) и хранятся в определённой кодировке;
   - Бинарные — содержат данные в «сыром» виде (.exe, .png, .mp3, .dat), предназначенном только для интерпретации и обработки программой, не человеком.
2. **Какая последовательность действий при работе с файлом?**
   1. Открыть файл (open());
   2. Выполнить нужную операцию;
   3. Закрыть файл (close()).
3. **Какие параметры необходимо указать при открытии файла?**
   Наименование файла (название или путь до него), режим доступа и кодировку.
4. **В чём разница между относительным и абсолютным путями к файлу?**
   - Относительный путь — относительно текущей рабочей директории;
   - Абсолютный путь — полный путь от корневой папки (C:\) до файла.
5. **Какие существуют режимы доступа для открытия файла?**
   - r — чтение (режим по умолчанию);
   - w — запись (перезаписывает содержимое файла);
   - a — добавление (в конец файла);
   - x — создание файла;
   - b — бинарный режим (rb; wb и т.д.);
   - t — текстовый режим;
   - + — чтение и запись.
6. **Какие методы существуют для чтения данных и в чём разница между ними?**
   - **read():** читает весь файл как одну строку;
   - **readline():** читает одну строку из файла;
   - **readlines()**: читает весь файл как список строк;
   - for **line** in **file**: построчное чтение без загрузки всего файла в память.
7. **В чём преимущество использования менеджера контекста?**
   with **open**(...) as **f**: менеджер контекста, который закрывает файл автоматически при возникновении исключения, что предотвращает утечки памяти.
8. **В чем отличия работы с текстовым и бинарным файлами?**
   - Текстовые файлы пишутся/читаются как строки и автоматически преобразуется согласно используемой кодировке;
   - Бинарные файлы работают с объектами типа **bytes**, не кодируются и используются для нетекстовых данных (изображения, архивы и т.д.).

9. **Как добавить и считать данные из бинарного файла?**
   Подключив файл в программу в бинарном режиме работы:
   with open('file.bin', 'wb') as f:
   f.write(data) # data — **bytes**
   with open('file.bin', 'rb') as f:
   data = f.read() # считывает как **bytes**

ВЫВОДЫ

Я получил практические навыки разработки и отладки программ, работающих с текстовыми файлами на языке Python.